销售系统需求建模例题

业务需求如下：

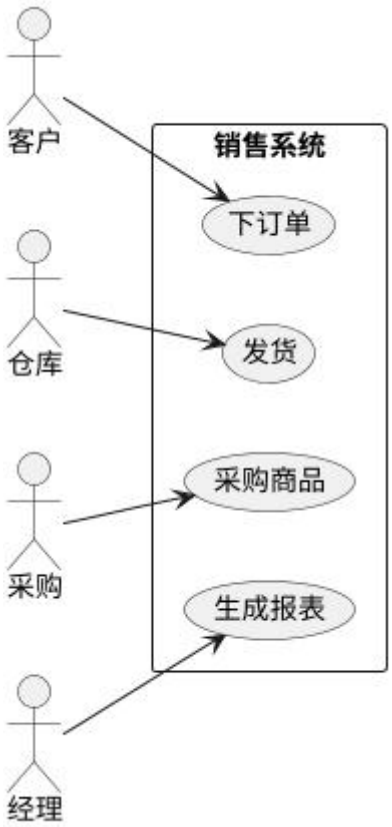
为提高销售流程的运行效率，开发销售系统：接受顾客订单后，查询库存记录，如果有货，则生成有货订单，更新库存记录; 仓库根据订单发货；如果无货，则生成缺货记录和缺货订单；采购部门根据缺货记录情况采购商品，待商品到货后更新库存记录，并查看是否有客户缺货订单，如有，则改为有货订单，仓库发货；系统根据订单记录，生成销售统计表报送经理。

请根据需求描述，建立用例模型，并创建类图、顺序图，状态图等分析模型。

一、用例模型

参与者（4 个）：**客户、仓库、采购、经理**

用例（4 个）：**下订单、发货、采购商品、生成报表**



用例 1：UC01 下订单

用例 ID：UC01
用例名称：下订单
参与者：客户（主参与者）
前置条件：客户进入销售系统，具备下单权限
后置条件

1. 分支 A（库存充足）：生成有货订单，库存数量更新，触发发货流程；
2. 分支 B（库存不足）：生成缺货订单与缺货记录，库存不变。

基本事件流

1. 客户提交订单（商品、数量）。
2. 系统查询库存记录。
3. 系统判断库存充足。
4. 系统生成订单记录。
5. 系统更新库存记录（扣减对应商品数量）。
6. 用例结束。

备选事件流

3a：库存不足，系统生成缺货记录，记录客户、商品、缺货数量。生成缺货订单，库存不修改。用例结束。

用例 2：UC02 发货

- **用例 ID：**UC02
- **用例名称：**发货
- **参与者：**仓库（主参与者）
- **前置条件：**系统存在待发货订单 / 待处理缺货补货记录
- **后置条件：**对应订单标记为已发货
- **基本事件流**

1. 仓库查看系统有货订单。
2. 仓库实物出库。
3. 仓库修改订单状态为【已发货】。
4. 用例结束。

- **备选事件流**

无

用例 3：UC03 采购商品

- **用例 ID：**UC03
- **用例名称：**采购商品
- **参与者：**采购（主参与者）
- **前置条件：**系统存在缺货记录
- **后置条件：**商品库存更新；系统自动检查是否存在对应的客户缺货记录
- **基本事件流**

    1. 采购查看系统缺货记录。
    2. 采购发起采购，完成商品到货。
    3. 系统更新库存记录（增加商品数量）。
    4. 系统检索对应的客户缺货订单。
    5. 如果存在缺货订单，更新为有货订单。
    6. 用例结束。

- **备选事件流**

    4a：无对应缺货订单：仅更新库存，不触发发货，用例结束。

    -

用例 4：UC04 生成报表

- **用例 ID：**UC04
- **用例名称：**生成报表
- **参与者：**经理（主参与者）
- **前置条件：**系统存在已保存的订单记录
- **后置条件：**生成销售统计表，报表可查看 / 导出
- **基本事件流**

    1. 经理触发生成销售统计表（或系统定期自动触发）。
    2. 系统读取已完成订单记录。
    3. 系统汇总数据，生成销售统计表。
    4. 系统展示报表给经理。
    5. 用例结束。

- **备选事件流**

    3a：无订单记录：系统提示暂无销售数据，报表为空，用例结束。

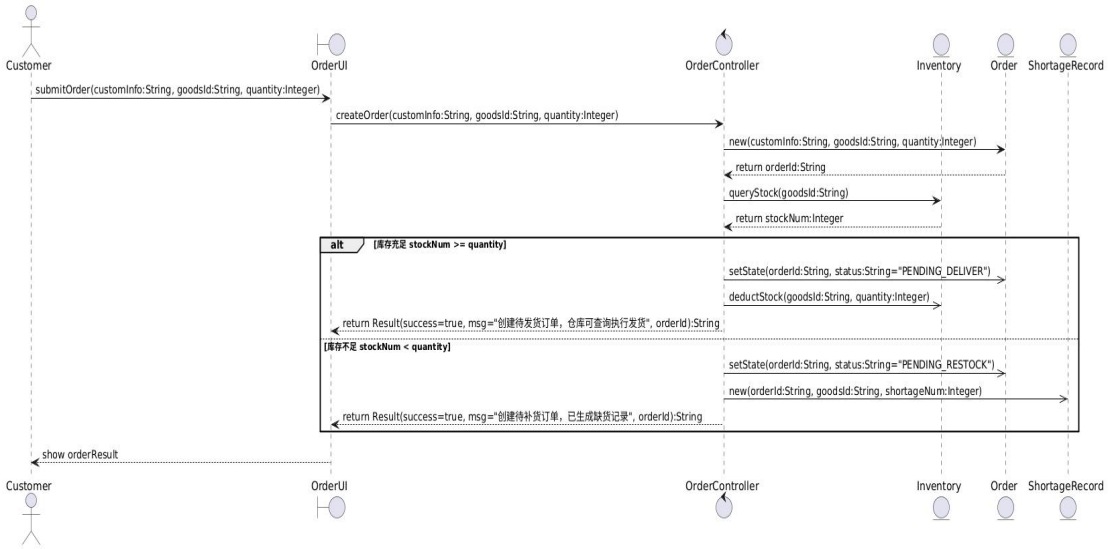
二、识别概念类

通过用例模型识别到的概念类：

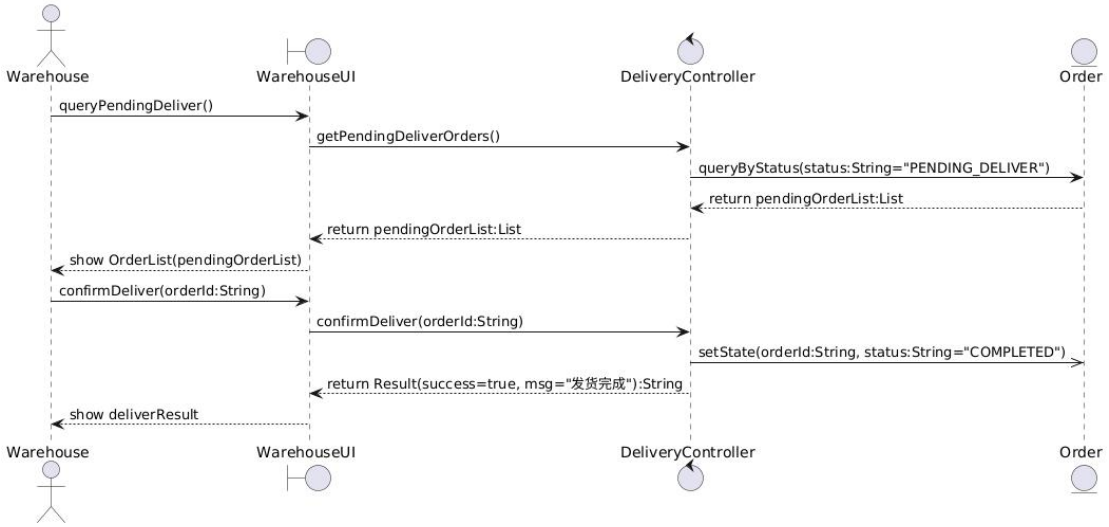
分类	类名	说明
边界类	OrderUI	客户下单界面
边界类	WarehouseUI	仓库操作界面
边界类	PurchaseUI	采购操作界面
边界类	ReportUI	经理报表界面
控制类	OrderController	下单业务控制
控制类	DeliveryController	发货业务控制
控制类	PurchaseController	商品入库、待补货订单状态更新
控制类	ReportController	销售报表统计
实体类	Order	订单实体，属性包括：订单号、商品、数量、客户、状态等
实体类	Inventory	库存实体，属性包括：商品编号、现有数量等
实体类	ShortageRecord	缺货记录实体，关联 Order 订单号、缺货商品、缺货数量
实体类	SalesStat	销售统计报表实体

三、分析模型

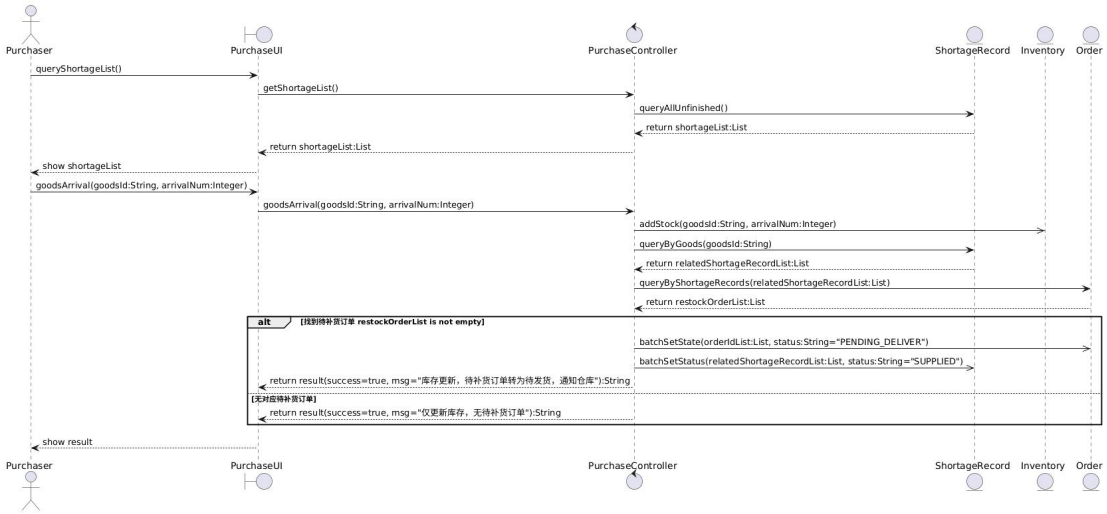
UC01 顺序图：



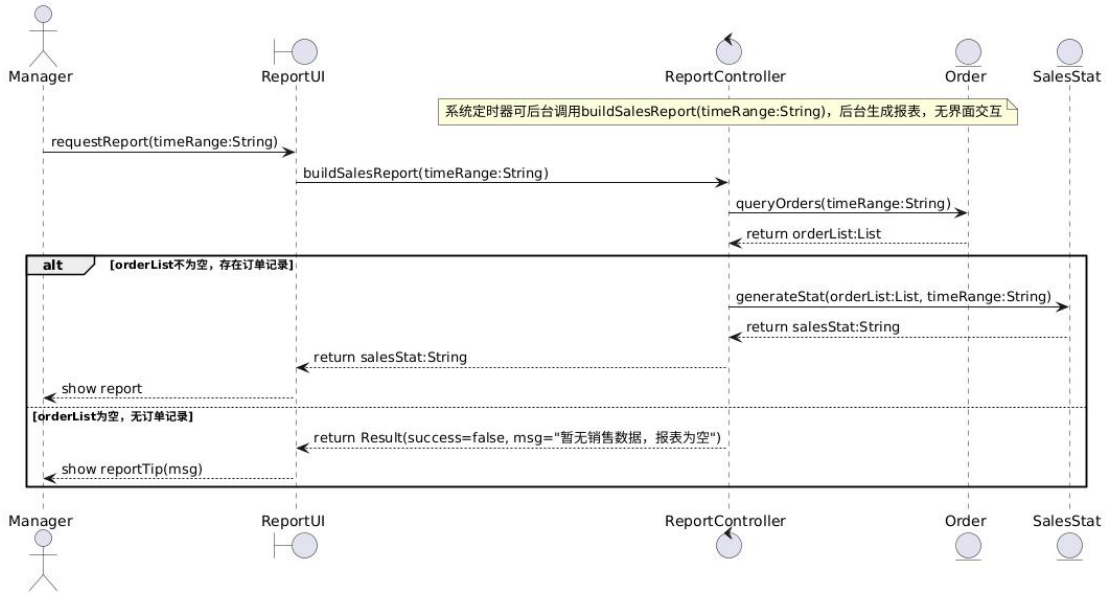
UC02 顺序图：



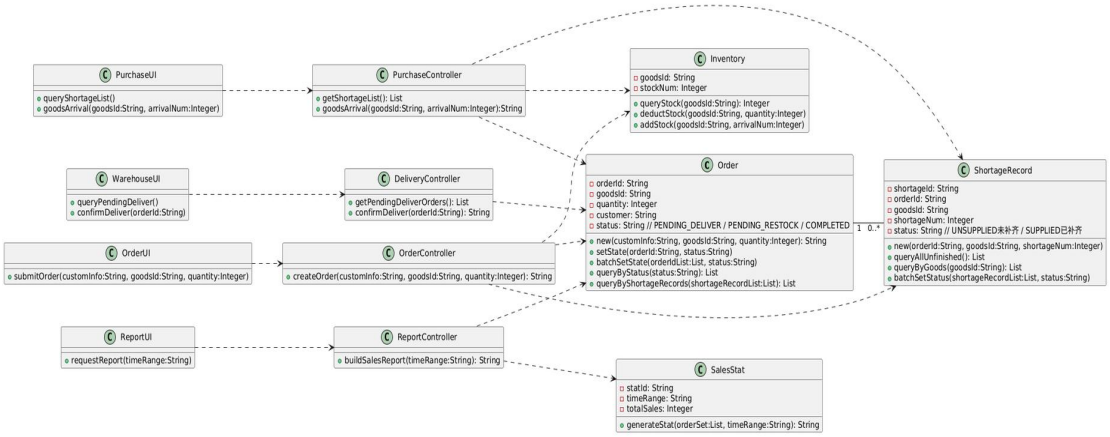
UC03 顺序图：



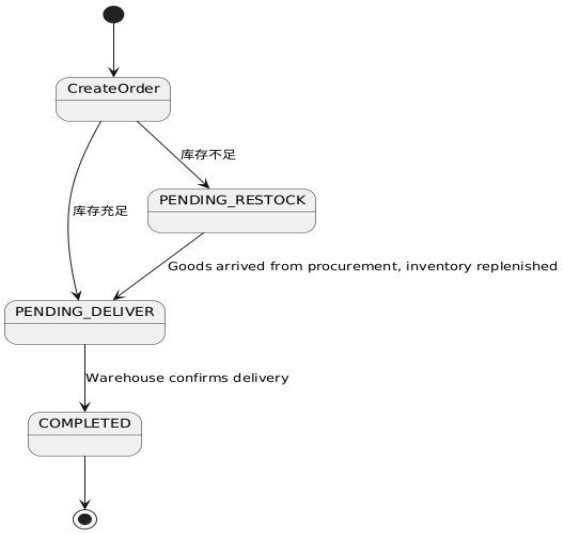
UC04 顺序图：



类图：



Order 类状态图：



附录：plantuml 代码

用例图代码

```
@startuml 销售系统用例图
left to right direction
actor 客户
actor 仓库
actor 采购
actor 经理

rectangle 销售系统 {
  usecase 下订单
  usecase 发货
  usecase 采购商品
  usecase 生成报表
}
客户 --> 下订单
仓库 --> 发货
采购 --> 采购商品
经理 --> 生成报表
@enduml
```

UC01 顺序图代码

```
@startuml
actor Customer as 客户
boundary OrderUI
control OrderController
entity Inventory
entity Order
entity ShortageRecord

客户 -> OrderUI: submitOrder(customInfo:String, goodsId:String, quantity:Integer)
OrderUI -> OrderController: createOrder(customInfo:String, goodsId:String, quantity:Integer)

OrderController -> Order: new(customInfo:String, goodsId:String, quantity:Integer)
Order --> OrderController: return orderId:String

OrderController -> Inventory: queryStock(goodsId:String)
Inventory --> OrderController: return stockNum:Integer

alt 库存充足 stockNum >= quantity
    OrderController ->> Order: setState(orderId:String, status:String="PENDING_DELIVER")
    OrderController ->> Inventory: deductStock(goodsId:String, quantity:Integer)
```

```
    OrderController --> OrderUI: return Result(success=true, msg="创建待发货订单，仓库可查询执行发货", orderId):String
else 库存不足 stockNum < quantity
    OrderController ->> Order: setState(orderId:String, status:String="PENDING_RESTOCK")
    OrderController ->> ShortageRecord: new(orderId:String, goodsId:String, shortageNum:Integer)
    OrderController --> OrderUI: return Result(success=true, msg="创建待补货订单，已生成缺货记录", orderId):String
end

OrderUI --> 客户: show orderResult
@enduml
```

UC02 顺序图代码

```
@startuml UC02 发货
actor Warehouse as 仓库
boundary WarehouseUI
control DeliveryController
entity Order

仓库 -> WarehouseUI: queryPendingDeliver()
WarehouseUI -> DeliveryController: getPendingDeliverOrders()
DeliveryController -> Order: queryByStatus(status:String="PENDING_DELIVER")
Order --> DeliveryController: return pendingOrderList:List
DeliveryController --> WarehouseUI: return pendingOrderList:List
WarehouseUI --> 仓库: show OrderList(pendingOrderList)

仓库 -> WarehouseUI: confirmDeliver(orderId:String)
WarehouseUI -> DeliveryController: confirmDeliver(orderId:String)
DeliveryController ->> Order: setState(orderId:String, status:String="COMPLETED")
DeliveryController --> WarehouseUI: return Result(success=true, msg="发货完成"):String
WarehouseUI --> 仓库: show deliverResult
@enduml
```

UC03 顺序图代码

```
@startuml UC03 采购商品
actor Purchaser as 采购
boundary PurchaseUI
control PurchaseController
entity ShortageRecord
entity Inventory
entity Order

采购 -> PurchaseUI: queryShortageList()
```

```
PurchaseUI -> PurchaseController: getShortageList()
PurchaseController -> ShortageRecord: queryAllUnfinished()
ShortageRecord --> PurchaseController: return shortageList:List
PurchaseController --> PurchaseUI: return shortageList:List
PurchaseUI --> 采购: show shortageList

采购 -> PurchaseUI: goodsArrival(goodsId:String, arrivalNum:Integer)
PurchaseUI -> PurchaseController: goodsArrival(goodsId:String, arrivalNum:Integer)

PurchaseController ->> Inventory: addStock(goodsId:String, arrivalNum:Integer)
PurchaseController -> ShortageRecord: queryByGoods(goodsId:String)
ShortageRecord --> PurchaseController: return relatedShortageRecordList:List
PurchaseController -> Order: queryByShortageRecords(relatedShortageRecordList:List)
Order --> PurchaseController: return restockOrderList:List

alt 找到待补货订单 restockOrderList is not empty
    PurchaseController ->> Order: batchSetState(orderIdList:List, status:String="PENDING_DELIVER")
    PurchaseController ->> ShortageRecord: batchSetStatus(relatedShortageRecordList:List,
status:String="SUPPLIED")
    PurchaseController --> PurchaseUI: return result(success=true, msg="库存更新，待补货订单转为待
发货，通知仓库"):String
else 无对应待补货订单
    PurchaseController --> PurchaseUI: return result(success=true, msg="仅更新库存，无待补货订单
"):String
end

PurchaseUI --> 采购: show result
@enduml

UC04 顺序图代码
@startuml
actor Manager as 经理
boundary ReportUI
control ReportController
entity Order
entity SalesStat

note over ReportController: 系统定时器可后台调用 buildSalesReport(timeRange:String)，后台生成报表，
无界面交互

经理 -> ReportUI: requestReport(timeRange:String)
ReportUI -> ReportController: buildSalesReport(timeRange:String)
```

```
ReportController -> Order: queryOrders(timeRange:String)
Order --> ReportController: return orderList:List

alt orderList 不为空，存在订单记录
    ReportController -> SalesStat: generateStat(orderList:List, timeRange:String)
    SalesStat --> ReportController: return salesStat:String
    ReportController --> ReportUI: return salesStat:String
    ReportUI --> 经理: show report
else orderList 为空，无订单记录
    ReportController --> ReportUI: return Result(success=false, msg="暂无销售数据，报表为空")
    ReportUI --> 经理: show reportTip(msg)
end
@enduml
```

类图代码

```
@startuml 销售系统类图
left to right direction

'==================== 实体类 ====================
class Order {
    - orderId: String
    - goodsId: String
    - quantity: Integer
    - customer: String
    - status: String // PENDING_DELIVER / PENDING_RESTOCK / COMPLETED
    + new(customInfo:String, goodsId:String, quantity:Integer): String
   + setState(orderId:String, status:String)
   + batchSetState(orderIdList:List, status:String)
    + queryByStatus(status:String): List
    + queryByShortageRecords(shortageRecordList:List): List
}

class Inventory {
    - goodsId: String
    - stockNum: Integer
    + queryStock(goodsId:String): Integer
    + deductStock(goodsId:String, quantity:Integer)
    + addStock(goodsId:String, arrivalNum:Integer)
}

class ShortageRecord {
    - shortageId: String
```

```
    - orderId: String
    - goodsId: String
    - shortageNum: Integer
    - status: String // UNSUPPLIED 未补齐 / SUPPLIED 已补齐
    + new(orderId:String, goodsId:String, shortageNum:Integer)
    + queryAllUnfinished(): List
    + queryByGoods(goodsId:String): List
    + batchSetStatus(shortageRecordList:List, status:String)
}

class SalesStat {
    - statId: String
    - timeRange: String
    - totalSales: Integer
    + generateStat(orderSet:List, timeRange:String): String
}

'==================== 边界类 ====================
class OrderUI {
    + submitOrder(customInfo:String, goodsId:String, quantity:Integer)
}

class WarehouseUI {
    + queryPendingDeliver()
    + confirmDeliver(orderId:String)
}

class PurchaseUI {
    + queryShortageList()
    + goodsArrival(goodsId:String, arrivalNum:Integer)
}

class ReportUI {
    + requestReport(timeRange:String)
}

'==================== 控制类 ====================
class OrderController {
    + createOrder(customInfo:String, goodsId:String, quantity:Integer): String
}

class DeliveryController {
    + getPendingDeliverOrders(): List
```

```
    + confirmDeliver(orderId:String): String
}

class PurchaseController {
    + getShortageList(): List
    + goodsArrival(goodsId:String, arrivalNum:Integer):String
}

class ReportController {
    + buildSalesReport(timeRange:String): String
}

'==================== 类间关系 ====================
OrderUI ..> OrderController
OrderController ..> Order
OrderController ..> Inventory
OrderController ..> ShortageRecord

WarehouseUI ..> DeliveryController
DeliveryController ..> Order

PurchaseUI ..> PurchaseController
PurchaseController ..> Inventory
PurchaseController ..> ShortageRecord
PurchaseController ..> Order

ReportUI ..> ReportController
ReportController ..> Order
ReportController ..> SalesStat

Order "1" -- "0..*" ShortageRecord
@enduml

Order 类状态图代码
@startuml
[*] --> CreateOrder
CreateOrder --> PENDING_DELIVER : 库存充足
CreateOrder --> PENDING_RESTOCK : 库存不足

PENDING_RESTOCK --> PENDING_DELIVER : Goods arrived from procurement, inventory replenished
PENDING_DELIVER --> COMPLETED : Warehouse confirms delivery
COMPLETED --> [*]
@enduml
```